Accessible Education Platform (AEP) – Final Report

Academic Group 9 · PGDM / PGDM(BM) 2025–27 · Maker Lab

Name	Roll No.
Adithiya M B	PGPBM-25-125
Kedar Phutane	PGP-25-098
Rameshta Vishwakarma	PGP-25-240
Samarpita Debnath	PGP-25-171
Shrey Srivastava	PGP-25-190

Table of Contents

Link to application 2

1. Design 2
   - 1.1 Problem in One Sentence 2
   - 1.2 Who We Designed For 2
   - 1.3 What Changes After Using AEP 3
   - 1.4 Core Design Principles 3
2. Requirements Table 4
3. Flow of Features 5
   - 3.1 Overall System Flow (End-to-End) 5
   - 3.2 Admin Flows 6
   - 3.3 Student Flows 6
     - 3.3.1 First Login and Profile Setup 6
     - 3.3.2 Reading a Lecture 7
   - 3.4 Teacher Flows 8
     - 3.4.1 Upload and Conversion 8
     - 3.4.2 Reviewing AI Output (Tier 2) 9
4. What Is Done, What Isn't, What's Next 9
   - 4.1 What Is Done (MVP Scope Achieved) 9
   - 4.2 What Is Partially Done 10
   - 4.3 What's For Future (Roadmap) 10
5. AI Usage, Benchmarks, and Testing 11
   - 5.1 Where We Use AI (and Where We Don't) 11
   - 5.2 Simplified Text Pipeline – Micro Benchmark 11
   - 5.3 System-Level Testing (Playwright + Gemini) 12
   - 5.4 Current Testing Status (At Final Submission) 13
6. What We Learned 13

Link to application

Here is the link to our application: https://accessible-education-v2-production.up.railway.app/login

1. Design

1.1 Problem in One Sentence

When a college teacher uploads a lecture file, a student with disabilities usually gets it in only one format, days later, or not at all.

Our goal: **one upload, many accessible formats, ready in minutes, without the student having to ask**.

1.2 Who We Designed For

We focused on three people. Think of them as real humans, not abstract "users."

- **Maya – 17-year-old student**
  - Deaf-Blind, has ADHD.
  - Uses a screen reader and Braille device every day.
  - Has 5–6 courses each semester, with weekly uploads.
  - Today, she needs 3 separate apps to read one lecture: one for Braille, one for captions, one for simplified text. None of these talk to each other or know her courses.
- **Professor Johnson – teacher**
  - Teaches ~120 students, 18 with disabilities.
  - Spends 5–6 hours per lecture creating captions, transcripts, and simplified notes manually.
  - Willing to do the work, but has no tool that scales.
- **Dr. Patel – admin**
  - Manages 8,000 students and 950 education plans in spreadsheets.
  - Raises IT tickets to do basic tasks like assigning a teacher to a course.

We designed the platform so that:

- A student like Maya sets her accessibility profile once.
- A teacher like Johnson uploads a file once.
- An admin like Dr. Patel runs the institution without begging IT for help.

1.3 What Changes After Using AEP

For each person, “before vs after” in very simple terms.

- **Student:**
  - Before: Needs to hunt for transcripts or audio, or simply cannot access content.
  - After: Logs in once, sets her profile (for example: Deaf-Blind + ADHD). Every lecture she sees already has the formats she needs: Audio, Transcript, Simplified, High-Contrast, Braille-ready. No extra requests, no waiting.
- **Teacher:**
  - Before: 5–6 hours per lecture for accessibility work.
  - After: Uploads once, reviews AI-generated simplified text for ~2 minutes, and clicks Approve. The system does the rest.
- **Admin:**
  - Before: Spreadsheets, manual enrolment, IT tickets.
  - After: Can:
    - Set up institute hierarchy.
    - Bulk enroll 3,000 students.
    - Assign teachers.
    - Export WCAG compliance reports directly from the UI.

1.4 Core Design Principles

We made three conscious design choices.

1. **No guessing**
   - Formats are chosen with fixed rules, not AI “magic.”
   - Example: If the profile says “Blind”, we pre-select Audio. If it says “ADHD”, we pre-select Simplified.
2. **Always show a clear state**
   - Content is never “mysteriously loading.” It is either:
     - Converting,
     - Published,
     - Not available, with a human-readable reason.
3. **Match real-life workflows**
   - Teachers think in “divisions”, not database IDs.
   - Students think in “formats they can actually use”, not in file extensions.
   - Admins think in “cohorts and compliance,” not raw tables.

2. Requirements Table

This table connects **user needs → system requirement → our implementation → current status**.

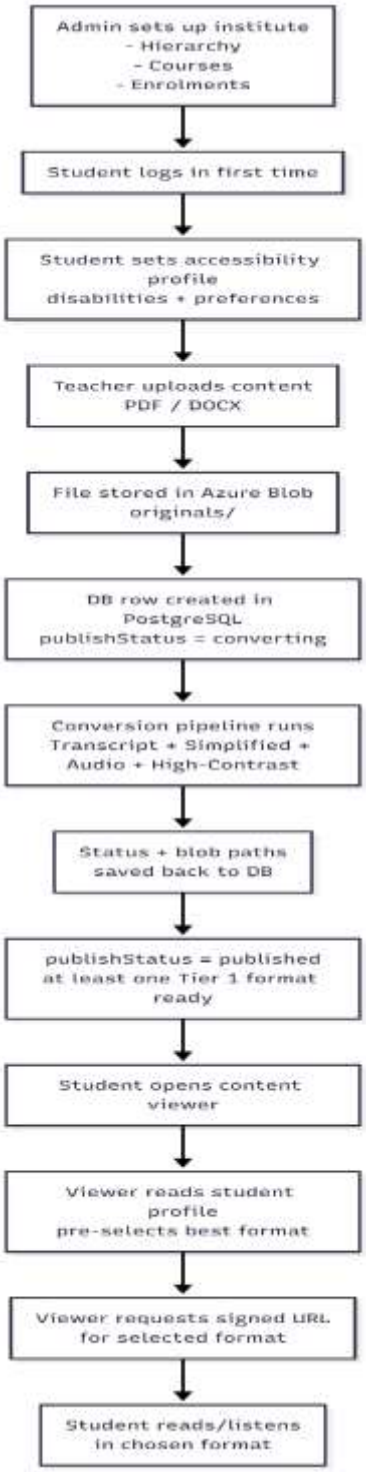
ID	User Need (plain)	Requirement	Implementation Summary	Status
R1	Student wants the right format without asking each time	Profile-based format defaults	Student profile stores disabilities and preferences; viewer pre-selects Audio/Simplified based on rules.	Done
R2	Teacher wants to upload once, not four times	Single upload → multi-format conversion	3-step upload modal; backend pipeline generates transcript, simplified text, audio script, high-contrast PDF.	Done (PDF/DOCX)
R3	Admin wants to manage courses and enrolments without IT	Admin hierarchy + enrolment tools	UI for institute hierarchy, teacher assignment, bulk enrolment, RBAC on all APIs.	Done
R4	Student should never see a blank or broken viewer	Safe fallback states	If conversion fails or is pending, formats are greyed out or marked "not available"; original file always accessible.	Done
R5	Teachers want to trust simplified text	Human review for risky AI outputs	Tiered system: "safe" formats auto-publish; Simplified and Braille require teacher approval.	Done
R6	System must store real data, not dummy strings	End-to-end real data path	PostgreSQL (Neon) + Drizzle ORM; Azure Blob for files; viewer fetches signed URLs at runtime.	Done (recent DB fix)
R7	System must handle failures honestly	Clear error pathways	Scanned PDFs mark all text formats as FAILED and show explicit warnings to teacher and student.	Done
R8	Support video and image content	Video/image conversion	Planned: FFmpeg + Whisper for video; Tesseract OCR for scanned PDFs/images.	Future
R9	Handle large PDFs and high load	Robust conversion infrastructure	Planned move from inline conversion to BullMQ + Redis job queue.	Future

ID	User Need (plain)	Requirement	Implementation Summary	Status
R10	Delegation for Tier 2 reviews	Share review load	Planned TA review delegation; multiple reviewers per teacher.	Future

3. Flow of Features

3.1 Overall System Flow (End-to-End)

This is the “life of a file” in our system.



3.2 Admin Flows

Admin Setup Flow – what the admin actually does.

What works **today**:

- Create hierarchical structure.
- Create courses and divisions.
- Assign teachers.
- Bulk enrol cohorts.
- RBAC enforcement on all admin actions.

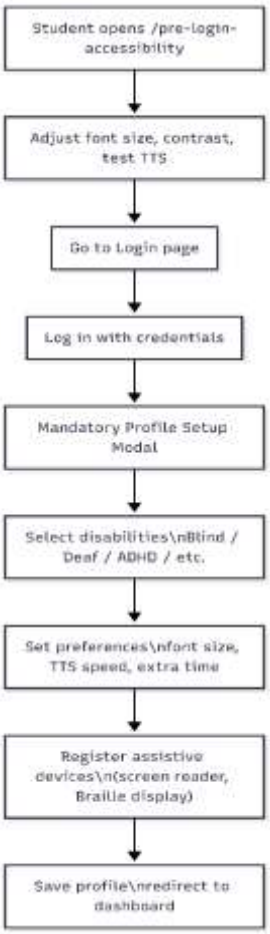
What is **not** built yet:

- Full analytics dashboards.
- Detailed per-student format readiness. (Planned for v2.1)

3.3 Student Flows

3.3.1 First Login and Profile Setup

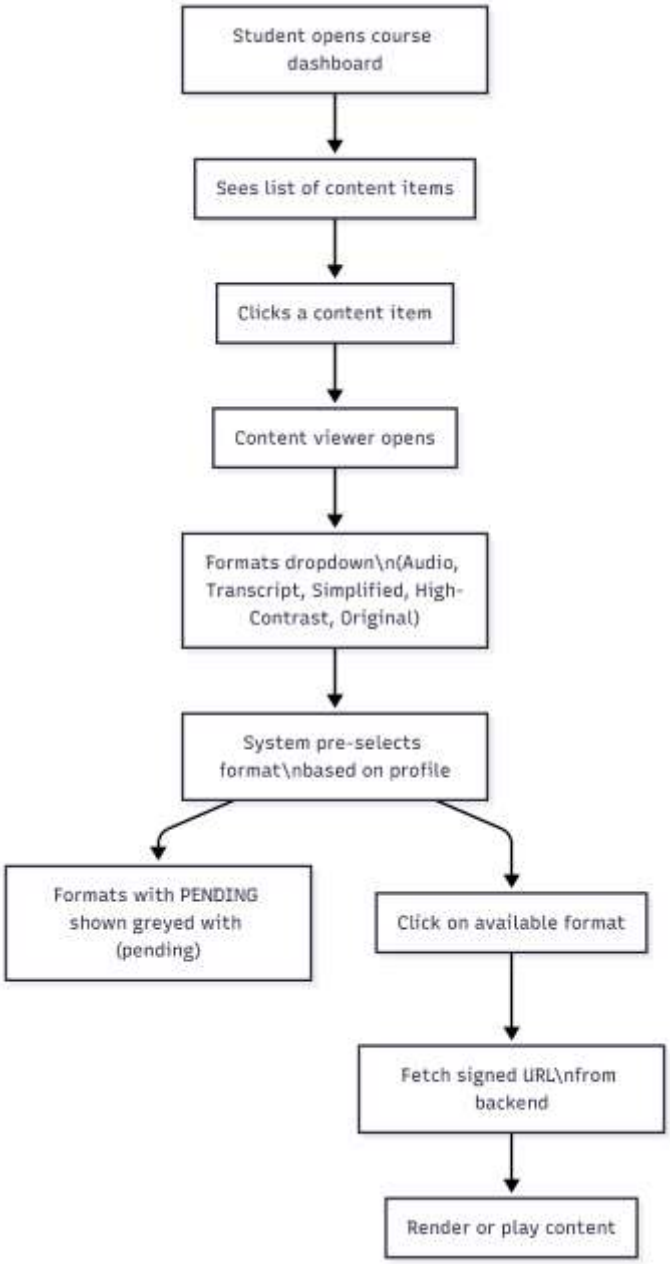
Following is the ideal flow of the first login and profile setup functionality:



Everything above is **implemented and working**, including "pre-login" accessibility configuration.

3.3.2 Reading a Lecture

Following is the ideal flow for the reading a lecture functionality:



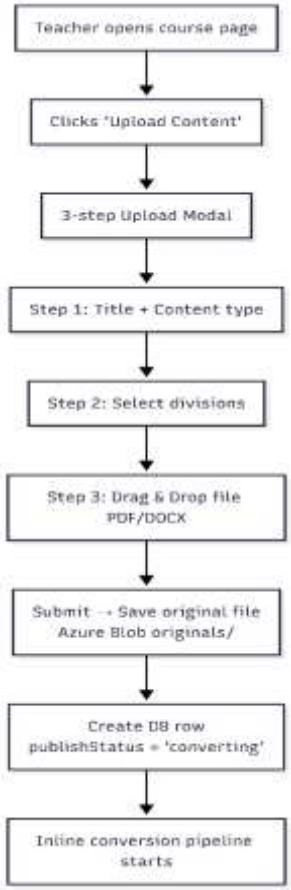
What works **today**:

- Profile-based default format.
- Clear state for each format: Available / Pending / Not available.
- Never shows a blank viewer: original file always accessible.

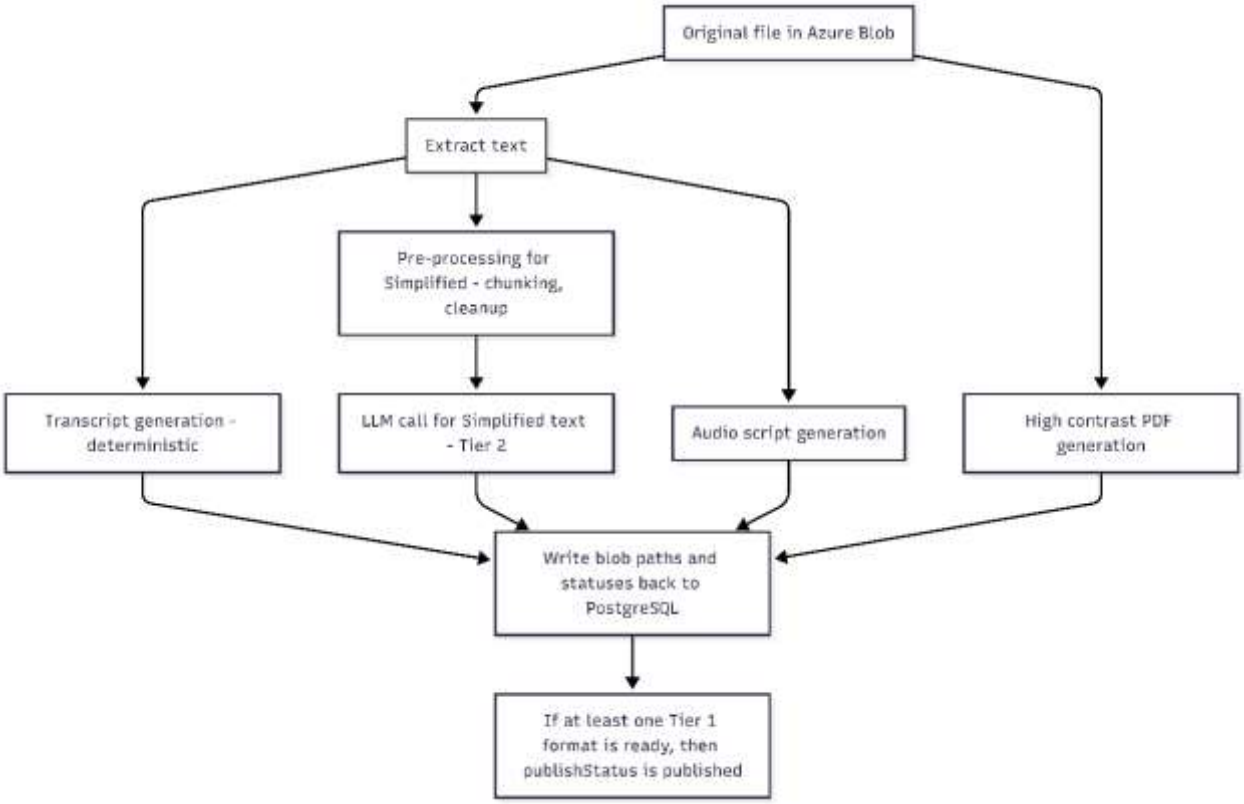
3.4 Teacher Flows

3.4.1 Upload and Conversion

Following is the ideal flow for the reading a lecture functionality:



The flow for the conversion pipeline itself is as follows:



What is **working now**:

- All DB writes converted from Mongo-style to Drizzle ORM for PostgreSQL.
- Conversion logs clearly show success for each step.
- Frontend now reads **real** DB state instead of hardcoded demo values.

3.4.2 Reviewing AI Output (Tier 2)

For Simplified Text (and future Braille), we use a **two-stage process**.

1. **Deterministic pre-processing**
    - Clean and split text.
    - Group into chunks.
    - Replace hard-to-read connectives.
2. **LLM simplification with strict rules**
    - Max 2 sentences per paragraph.
    - Use words a 14-year-old can understand.
    - Keep all technical terms, explain them in brackets.
    - No extra information allowed.

Teacher sees a side-by-side view and can Approve or Reject each chunk.

---

4. What Is Done, What Isn't, What's Next

4.1 What Is Done (MVP Scope Achieved)

End-to-end pipeline (PDF/DOCX)

- Admin setup, course hierarchy, enrolment.
- Student profile and accessibility defaults.
- Teacher upload and conversion for:
    - Transcript,
    - Simplified text (with review),
    - Audio script,
    - High-contrast PDF.
- Student viewer with profile-based defaults and safe fallback states.

RBAC and security

- Role-based access control at API layer: Admin, Teacher, Student.
- Students cannot use teacher/admin endpoints; enforced via 403 responses (not only UI hiding).

Failure handling

- Scanned PDFs:
    - Detect “no text” and mark all text formats as FAILED.
    - Show clear warnings and tooltips.
- Large PDFs:
    - Known timeout risk documented; roadmap fix defined (job queue).

Testing

- Playwright-based browser testing via Gemini CLI (MCP).
- 54 test areas covering auth, admin, teacher, student, ARIA, and full flows.
- Major bugs (Mongo-style writes, missing API calls) identified and fixed.

4.2 What Is Partially Done

- **Video and image support:**
    - Currently treat MP4 and images like “no text” PDFs: original file is accessible; derived formats are FAILED.
- **Large-document performance:**
    - Works well for documents up to ~12 pages.
    - May time out beyond ~30 pages; fix planned via job queue.
- **Per-student readiness:**
    - Content is marked “published” once Tier 1 formats exist, not once *each* individual student’s required format is ready.

4.3 What’s For Future (Roadmap)

We explicitly wrote a v2.1 roadmap.

1. **Job queue for conversion**
    - Move conversion from inline Node.js functions to BullMQ + Redis.
    - Handles large PDFs and concurrent uploads reliably.
2. **Video transcription (FFmpeg + Whisper)**
    - Extract audio, then text from MP4 videos.
    - Unlocks Transcript/Simplified/Audio for video content.
3. **OCR for scanned PDFs and images (Tesseract.js)**
    - Recover text from non-searchable PDFs and images.
    - Critical for Indian colleges where many files are scans.
4. **Per-student publish readiness check**
    - Only show “published” if *the student’s* minimum required format is ready.

    - Prevents “published but unusable” situations for complex profiles like Deaf-Blind.
5. **TA delegation for Tier 2 review**
    - Let teachers assign TAs to help review simplified texts and Braille outputs.
    - Spread workload across the teaching team.

---

5. AI Usage, Benchmarks, and Testing

5.1 Where We Use AI (and Where We Don’t)

We use an LLM only for **Simplified Text**. Everything else is deterministic.

- **Deterministic (no AI):**
    - Transcript extraction (text-based PDFs and DOCX).
    - High-contrast PDF transformation via pdf-lib.
    - Audio script generation based on the transcript and Web Speech API for playback.
- **LLM-based:**
    - Simplified text chunks using a carefully designed system prompt.
    - We control chunk size and structure before calling the model.
    - Teachers must approve simplified text before it reaches students.

We deliberately **do not** use AI for:

- Access control decisions.
- Role-based permissions.
- Which formats to publish first.
- Any grading or evaluation.

All those are rule-based and transparent.

5.2 Simplified Text Pipeline – Micro Benchmark

We evaluated the Simplified Text pipeline on academic content (e.g., “Basics of C++”).

- Average chunk size: ~3 sentences, up to 60 words per chunk.
- Model output:
    - Reduced complexity of sentences.
    - Preserved key terms with bracketed explanations.
    - Occasionally added creative analogies not present in the source.

Outcome:

- For most chunks, the simplified version was easier for a 14–16 year old to read.

- For some tricky content (e.g., neural networks), the model invented analogies not in the original, which could mislead students.

This is the main reason for the teacher review gate.

5.3 System-Level Testing (Playwright + Gemini)

We used **Playwright tests triggered via Gemini CLI** to simulate real user behaviour.

Coverage:

- 54 major test areas across:
    - Auth & session.
    - Admin hierarchy and user management.
    - Teacher upload and conversion.
    - Student viewer and formats.
    - ARIA and keyboard-only navigation.
    - End-to-end flows.

Sample process:

1. Write or generate a Playwright test with Gemini MCP.
2. Run against the live deployment.
3. Inspect DOM, network calls, and console logs.
4. Cross-check with database state (Neon + Drizzle).

Two concrete examples of bugs discovered and fixed:

- **Conversion pipeline bug:**
    - Code still used findByIdAndUpdate() style (Mongo) on a PostgreSQL DB.
    - No DB writes were actually happening; UI showed demo data.
    - Fix: Rewrote all queries using Drizzle ORM and added explicit try/catch + console.error logging.
- **Hierarchy action buttons bug:**
    - “Edit / Add Child / Retire Node” showed success messages but made no API calls.
    - Fix: Wired buttons to real backend endpoints and removed fake success toasts.

5.4 Current Testing Status (At Final Submission)

From the mid-review test summary (updated):

Area	Tests	Pass	Partial	Fail (open)
Auth & Session	10	9	0	1 resolved
Admin: Hierarchy	5	4	1	0
Admin: Users & Enrolment	8	6	1	1 resolved
Teacher: Upload & Conversion	7	5	1	1 resolved after ORM fix
Student: Viewer & Formats	8	5	2	1 resolved after ORM fix
ARIA & Keyboard	10	9	1	0
End-to-End Flow	6	4	1	1 resolved after ORM fix
Total	54	42	7	5 (resolved in code post-Round 2)

In later runs, some test failures were due to temporary **Neon database connectivity issues**, not application logic. Once the DB was reachable, tests for those flows passed.

---

6. What We Learned

Three key lessons we want to highlight.

1. **A correct-looking UI does not mean a correct system.**
   We had a fully "working-looking" viewer while the DB stored {} for available formats. The real state only appeared when we queried the database and checked server logs.

2. **Accessibility is about failure states as much as success states.**
   For a scanned PDF, the most honest behaviour is:

   - Tell the teacher clearly that conversion failed.
   - Tell the student clearly what is and isn't available.
   - Still give access to the original file.

3. **AI needs boundaries, not freedom.**
   Without constraints, LLMs will invent "helpful" examples. For students who get graded on that content, this is dangerous. Our Tier 2 review process and prompt design are direct responses to this.